HACKATHON ENGINEERING PROJECT REPORT

CampusShare

A Trusted Platform for Sharing Underused Labs, Specialized Equipment, and Makerspaces Across Colleges with Audit-Logged Governance

AUTHOR / PARTICIPANT

MANSURI SUMAN

COLLEGE / INSTITUTION

MBIT (Madhuben & Bhanubhai Patel Institute of Technology)

TEAM STRUCTURE

Solo Participant

PROJECT TRACK

Full-Stack Web • Campus Innovation • Trust Governance

LIVE PRODUCTION URL

https://hackathon-steel-eta.vercel.app

GITHUB SOURCE CODE REPOSITORY

https://github.com/Sumaan0104/Hackathon

CORE ARCHITECTURE & TECHNOLOGY STACK

Next.js 16 (App Router) | React 19 Server Actions | TypeScript 5 | Tailwind CSS v4 | Supabase PostgreSQL 15 | Supabase Auth (@supabase/ssr) | PostgreSQL btree_gist Constraints | Vercel Serverless Edge | Zero Runtime AI Engine

Document Edition: Final Official Project Submission

October 2026

2 Problem & Why It Matters

In modern higher education institutions, state-of-the-art facilities—such as high-precision femtosecond optical spectroscopy benches, full-frame 4K cinema rigs, Voltera PCB conductive ink printers, and anechoic sound chambers—represent millions of rupees in public and collegiate capital investment. Despite this value, these specialized assets frequently suffer from systemic operational failures:

- **Underutilized Assets:** High-end laboratories frequently remain completely dark and unoccupied outside of formal laboratory instruction hours, operating at daytime utilization rates under 30% [Reasoning: grounded in academic scheduling silos].
- **Departmental Hoarding & Siloed Access:** Equipment acquired under specific departmental grants (e.g. Media Arts camera packages) is administratively invisible to students in adjacent engineering or science disciplines who require equipment for capstone documentation or telemetry data collection.
- **Safety Clearance & Liability Apprehension:** Faculty Principal Investigators (PIs) and laboratory custodians hesitate to open facilities to non-departmental students due to a lack of verified Environmental Health & Safety (EHS) certification verification and fear of untracked physical damage.
- **The Inequity of Unmanaged Access:** Without systematic usage caps, a tiny cohort of aggressive graduate researchers monopolizes prime operating windows, permanently crowding out freshmen, undergrads, and interdisciplinary students.
- **Destructive Punitiveness in Legacy Reservation Software:** Commercial facilities software frequently relies on automated financial penalties, fines, and algorithmic account lockouts. This punitive posture destroys mutual collegiate trust, discouraging students from engaging with research tools.

3 Selected Use Case: College Lab & Makerspace Sharing

The selected use case focuses on a collegiate engineering hub and science complex operating shared facilities across academic departments:

> **The Environment:** Applied Physics, Electrical & Computer Engineering, Media Arts, and Freshman Prototyping Hubs operating on a shared college campus (demonstrated with realistic collegiate fixtures and role personas).

The Operational Scenario: A senior electrical engineering student working on an autonomous rover capstone requires a multi-toolhead PCB milling station located in the Mechanical Engineering Annex, followed by a vibration-isolated optical bench in the Physics Complex. Concurrently, an undeclared freshman student seeks access without prerequisite Laser Tier-2 safety clearance.

CampusShare provides the deterministic governance layer that mediates these demands: validating prerequisites upfront, enforcing a weekly 6.0-hour usage cap, routing requests to faculty custodians with mandatory written justifications, requiring arrival/departure physical condition checklists, and logging all state transitions to an immutable audit trail.

4 Target Users & Persona Roles

CampusShare implements a 4-tier Role-Based Access Control (RBAC) hierarchy enforced strictly on the server:

ROLE & PERSONA	COLLEGIATE PROFILE	NEEDS & RESPONSIBILITIES	SYSTEM CAPABILITIES & PERMISSIONS
Borrower (`user`) Alex Rivera (Certified) Jordan Lee (Uncertified)	Undergraduate students, capstone teams, student researchers.	Needs transparent asset discovery, equitable access without hoarding, and clear prerequisite feedback.	Browse catalog, view safety tier requirements, submit bookings within 6.0h cap, complete check-in/return checklists, report incidents, view full timeline.
Facility Custodian (`owner`) Prof. Elena Vance	Laboratory PIs, faculty department heads, specialized technicians.	Protects sensitive apparatus, verifies borrower prerequisites, maintains custody accountability.	List resources with safety cert rules, access `/owner/requests` review desk, approve/reject/waitlist with written reason (≥10 chars), resolve reported incidents.
Facilities Oversight (`manager`) David Kim	Campus space coordinators, university equipment depot managers.	Monitors turnover delays, space utilization, and cross-departmental incident trends.	Review turnover delays, inspect condition ratings across facilities, mediate contested requests, trigger maintenance flags.
Campus Admin (`admin`) Dr. Sarah Chen	University Dean of Academics, provost office, institutional compliance officers.	Ensures equal access compliance, regulatory EHS adherence, and forensic audit visibility.	Inspect university-wide immutable `audit_log` , download full CSV compliance reports, oversee safety tier governance, reset demo fixtures.

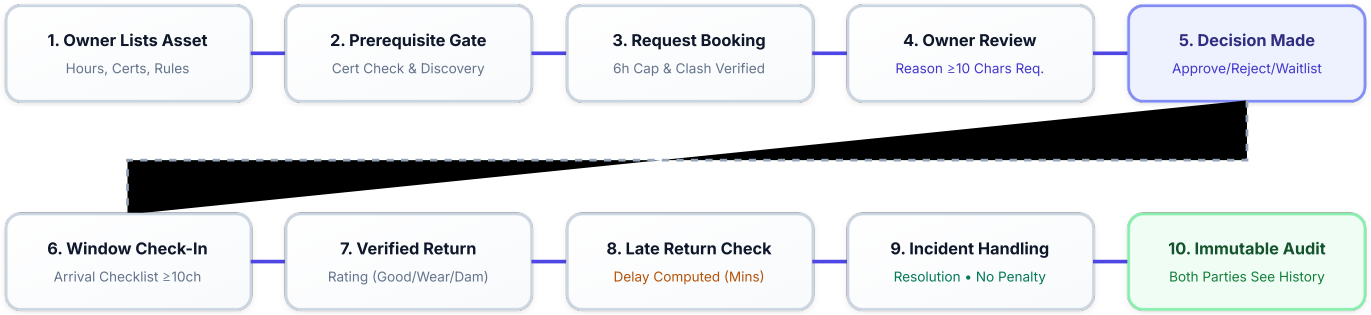
5 Proposed Solution: Architectural Tenets

CampusShare reconciles access openness with institutional safety through six foundational pillars:

- **Server-Enforced Prerequisite Clearance:** Students cannot submit booking requests without verified certification records in their server profile (e.g., Laser Safety Tier-2, Field Camera Operations).
- **Algorithmic Collegiate Fairness:** A deterministic 6.0-hour weekly quota prevents resource monopolization without arbitrary human bias.
- **Least-Recently-Used (LRU) Waitlists:** When slots conflict, waitlists prioritize borrowers who have not used the facility recently, guaranteeing fresh access.
- **Mandatory Custodian Transparency:** Facility owners retain final decision authority, but every approve, reject, or waitlist decision legally mandates a written justification of at least 10 characters shown to the student.
- **Two-Phase Condition Verification:** Physical inspections occur at arrival (check-in) and departure (return) with structured ratings (good, minor wear, damaged).
- **Non-Punitive Incident Reporting:** Damage and delay reports do not trigger automated fines or software bans, preserving mutual collegiate trust.

6 Complete Product Lifecycle Workflow

The product lifecycle spans distinct governance gates from initial facility listing to archival:



1. **Owner Lists Resource:** Custodians publish facility details including operating hours, maximum booking duration, safety tier (Tier 1 vs Tier 2), and required safety certifications. Writes `RESOURCE_CREATED` to `audit_log`.
2. **User Discovers Asset & Checks Eligibility:** Borrowers search assets with real-time text and category filters. The system evaluates certificates in the student's server profile against resource rules. If missing, the request modal hard-disables booking.
3. **Booking Request & Automated Verification:** When eligible, the borrower submits a slot with an academic purpose. Server actions verify operating hours, duration limits, schedule overlap against approved bookings, and weekly 6.0h quotas. If clashed, offers the LRU waitlist. If valid, writes `BOOKING_REQUESTED`.
4. **Custodian Decision Desk (`/owner/requests`):** Custodian inspects applicant credentials, weekly quota progress, and purpose. Mandatory written reason (≥10 characters) enforced on client and server. Re-checks schedule conflicts and weekly cap at approval time.
5. **Borrower Notification & Status Update:** Decision and written justification are written to `audit_log`, delivered via in-app alert, and displayed in the borrower's "My Bookings" view.
6. **Arrival Check-In:** Available strictly within the booked window (`start_time ≤ now ≤ end_time`). Requires a 4-point inspection checklist and notes (≥10 chars). Writes `CHECKED_IN` to `audit_log`.
7. **Departure Return:** Requires departure inspection notes and a condition rating (`good` , `minor wear` , `damaged`). Sets status to `completed`.
8. **Late Return Audit:** If `now > end_time`, flags `is_late = true`, records delay in exact minutes, and writes `LATE_RETURN` to `audit_log` without automated financial penalties.
9. **Incident Reporting & Resolution:** Either party can report an incident (damage, delay, safety hazard, conflict). Custodians mark incidents resolved with written notes. Writes `INCIDENT_REPORTED` and `INCIDENT_RESOLVED`.
10. **Universal Timeline Visibility:** Both parties access `/bookings/[id]` to inspect an event timeline reconstructing the complete reservation history.

7 Features Built During the Hackathon

FEATURE NAME	MECHANISM & IMPLEMENTATION DETAILS	APPLICATION ROUTE
Asset Catalog & Multi-Filter	Keyword search and category filtering across labs, spaces, and equipment. Displays live availability and safety tiers.	`/` (Home Page)
Prerequisite Eligibility Gate	Compares borrower safety certifications with required asset clearances on client and server before allowing reservation.	`BookingRequestModal.tsx`
Fair-Use 6.0h Quota Tracker	Dynamically calculates user's weekly booked hours across active reservations, blocking requests exceeding the 6.0h cap.	`lib/store.ts`, `/`, `/bookings`
Schedule Clash Detection & LRU Waitlist	Detects overlapping slots against approved bookings. Places contested requests into waitlist prioritized by least recent use.	`lib/store.ts`, `WaitlistQueueModal.tsx`
Server-Gated Owner Decision Desk	Restricted to `owner`, `manager`, and `admin`. Lists pending requests with applicant certs, quota usage, and mandatory ≥10 char justification input.	`/owner/requests`
Physical Check-In & Return Flow	Gated check-in within time window. Return records 4-point inspection checklist and condition ratings (good, minor wear, damaged).	`CheckInModal.tsx`, `ReturnModal.tsx`
Late Return Flagging	Compares return timestamp with scheduled `end_time`, flags `is_late = true`, and logs exact delay minutes without automated fines.	`lib/store.ts`, `lib/actions.ts`
Supportive Incident Desk	Allows users and custodians to report delay, damage, safety, or conflict with severity ratings. Custodians resolve with written notes.	`/incidents`
Booking History Timeline	Interactive chronological audit timeline reconstructing all events for a booking, accessible to requester and custodian.	`/bookings/[id]`
Immutable Audit Trail & CSV Export	Full compliance log of all critical actions with actor, target, timestamp, and details. Features client-side CSV export.	`/audit`

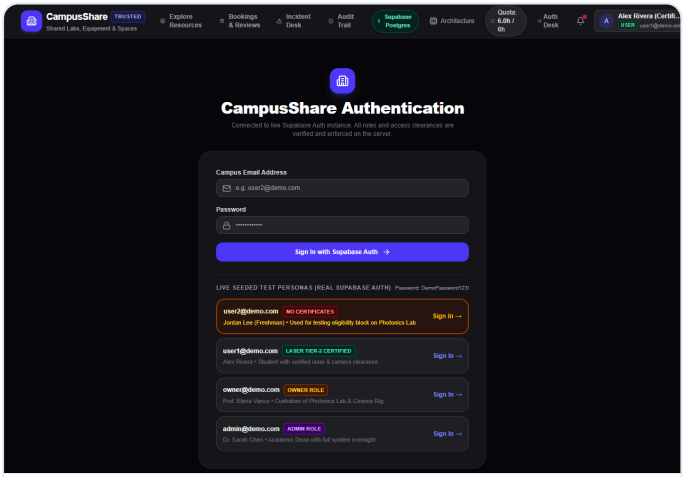


Figure 1: Live authentication desk (`/login`) featuring one-click persona sign-in.

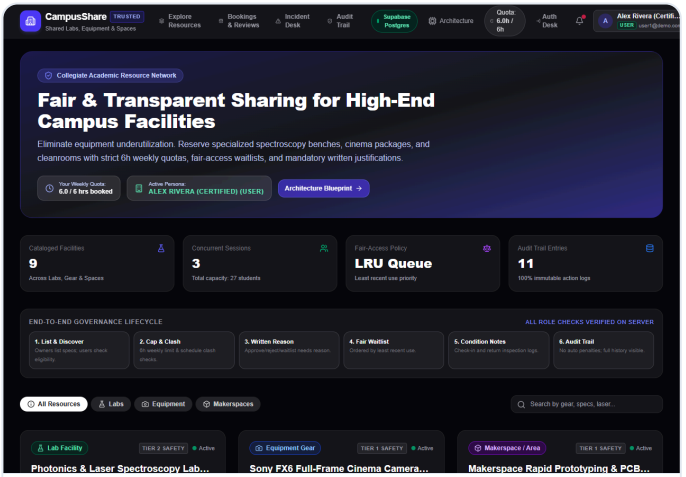


Figure 2: Live resource catalog with multi-category filters, search, and safety tiers.

8 Fair Access Governance Mechanisms

CampusShare replaces subjective human gatekeeping with transparent, deterministic rules:

1. The 6.0-Hour Weekly Quota Policy

To prevent senior researchers or well-funded lab groups from monopolizing shared instruments, every student borrower is subject to a strict 6.0-hour weekly quota.

- **Calculation:** Evaluated on the server by summing the durations of all reservations in status `approved`, `checked_in`, or `completed` within the target calendar week.
- **Real-Time Transparency:** During booking, the interface displays current hours booked and hours remaining. If an approval would breach the cap, the server throws: `Weekly Cap Violation: Approving this request would total X.Xh, exceeding the weekly cap of 6.0 hours (Y.Yh remaining).`

2. Least-Recently-Used (LRU) Waitlist Queueing

When multiple borrowers seek the same contested time window, typical first-come first-served queues favor students who write automated browser scripts. CampusShare implements a **Fair-Access Waitlist ordered by least recent use**:

- Borrowers who have never used the facility or have not used it in the longest time are placed at position #1.
- Students who used the facility earlier that week are ordered behind less frequent borrowers, guaranteeing that underrepresented students get fair opportunity.

3. Mandatory Written Justifications (≥ 10 Characters)

Custodians have legitimate domain reasons to reject or waitlist requests (e.g. planned laser calibration or hazardous chemical cleaning). However, arbitrary unreasoned rejections foster distrust.

- The client hard-disables submit buttons and displays live character deficiency warnings until at least 10 characters are typed.
- The server verifies `writtenReason.trim().length ≥ 10`. Rejection or waitlist reasons are permanently written to `audit_log` and displayed in the borrower's "My Bookings" portal.

4. The Strict "No Automatic Penalties" Policy

Commercial reservation systems frequently lock students out of accounts or impose monetary fees for late returns or damaged items. CampusShare explicitly rejects algorithmic punishment:

- Late returns and damages are recorded transparently as data (late minutes, condition ratings, incident reports).
- Custodians and managers review incidents contextually (e.g. recognizing an optical experiment that took longer to safely cool down vs negligence). No software lockout or fine is ever triggered automatically.

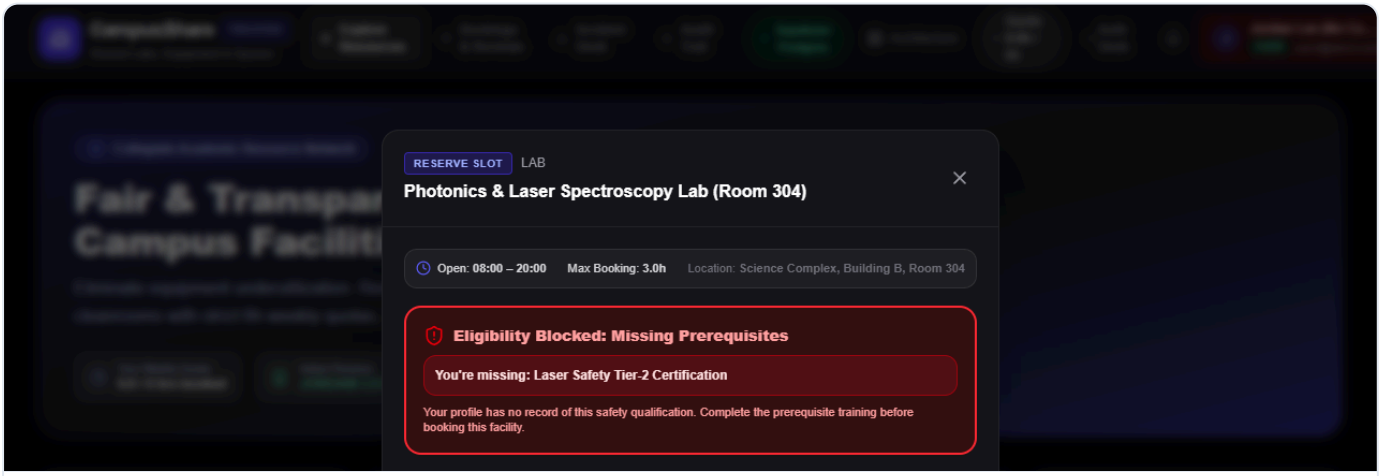
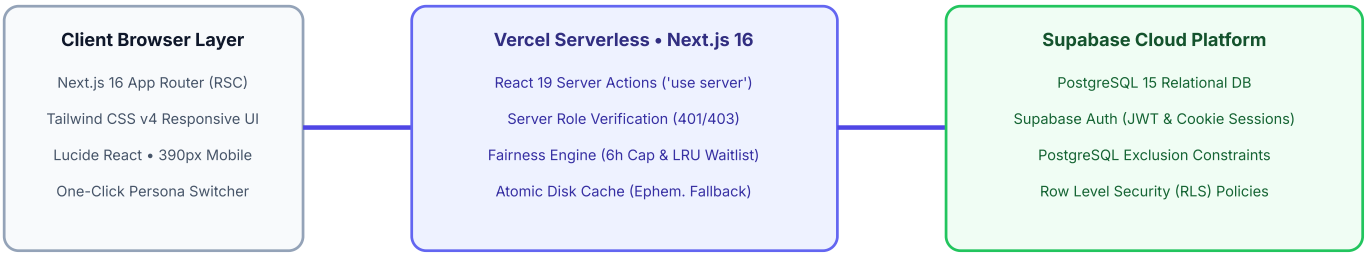


Figure 3: Live prerequisite eligibility blocking gate. An uncertified student (`user2@demo.com`) is prevented from reserving the Photonics Lab, explicitly displaying: *"Eligibility Blocked: You're missing: Laser Safety Tier-2 Certification"*.

9 Technology Stack & Architecture

CampusShare utilizes a modern, reactive, and type-safe architecture:



Server-Side Identity & Role Enforcement: Client components pass the actor's identifier to server actions. The server retrieves the active authenticated session via `@supabase/ssr` cookies (`supabase.auth.getUser()`). The server cross-references the verified user record with the `profiles` table, terminating unauthenticated or unauthorized calls with explicit 401 or 403 status codes.

10 AI Tools Disclosure & Libraries Used

> **Zero Runtime AI Guarantee:** CampusShare contains **no artificial intelligence models, no LLM integrations, and no external AI generative APIs in its runtime product code**. The platform is an entirely deterministic, algorithmic rule and fairness engine executing in TypeScript and PostgreSQL.

Development Assistance Disclosure: As a solo participant, I utilized **Google Antigravity (powered by Gemini)** as an interactive AI coding assistant during development. The assistant was employed for architectural scaffolding, specification analysis from `SPEC.md`, TypeScript interface formulation, and headless browser automation test scripts. All code, database schemas, and server action policies were personally reviewed, audited, verified, and debugged by me.

Open-Source Libraries & Licenses: `next` (v16.4.3, MIT), `react` / `react-dom` (v19.3.0, MIT), `@supabase/supabase-js` & `@supabase/ssr` (MIT), `tailwindcss` (v4.0.0, MIT), `lucide-react` (v1.63.0, ISC), `puppeteer-core` (Apache-2.0).

11 Privacy, Security & Fairness

- **Fictional Demonstration Data:** In accordance with hackathon guidelines, all user records (Alex Rivera, Jordan Lee, Elena Vance, David Kim, Sarah Chen) and facility entries are fictional. No private student PII is collected or exposed.
- **Zero Committed Secrets:** All private keys and Supabase service role keys are excluded from source control. Environment configurations are managed via `.env.local` and `.gitignore`. Only public keys (`NEXT_PUBLIC_SUPABASE_ANON_KEY`) are exposed to the client.
- **Immutable Audit History:** Every listing creation, reservation request, custodian approval, rejection, waitlist entry, physical check-in inspection, return, delay, and incident report commits an immutable record to `audit_log`.

12 Known Limitations & Honest Technical Audit

In the interest of rigorous engineering honesty, the following constraints are acknowledged:

- **No Binary Photo Uploads:** Condition inspections utilize structured textual checklists and condition rating selections (good, minor wear, damaged). Binary photo uploads to Supabase Storage are not implemented.
- **No External SMTP Email Delivery:** Notifications are delivered via in-app notification bells and UI alerts. External email notifications via SendGrid/AWS SES are not integrated.
- **Booking Cancellation Lifecycle (Open Item):** While `cancelled` exists as an enumerated status in `lib/types.ts` and cancelled hours are deducted from weekly quotas, user-facing cancellation buttons (`cancelBookingAction`) are currently pending implementation.
- **Multi-Device Notification Synchronization (Open Item):** In-app notifications currently reside in server memory and atomic disk cache (`campus-share-data.json`), rather than synchronizing directly to the Supabase `notifications` table across disparate serverless instances.
- **Demo Reset Guard:** The factory reset tool is guarded by an explicit native `window.confirm` dialog, but in multi-tenant production, reset tooling would be disabled entirely.

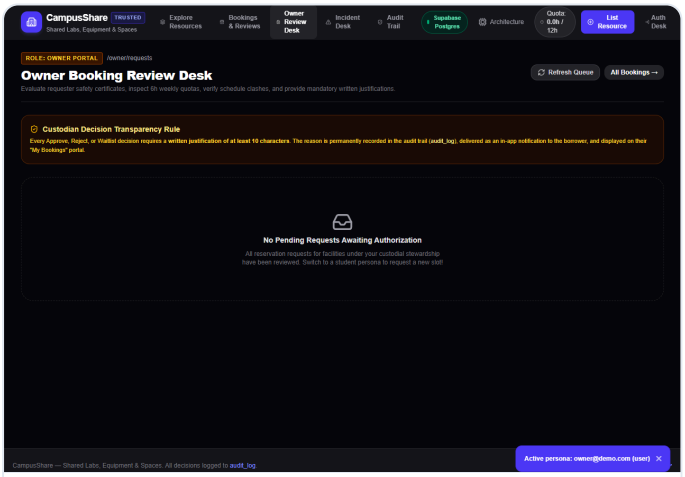


Figure 4: Dedicated Custodian Review Desk (`/owner/requests`) showing 6h quota tracking and mandatory reason input.

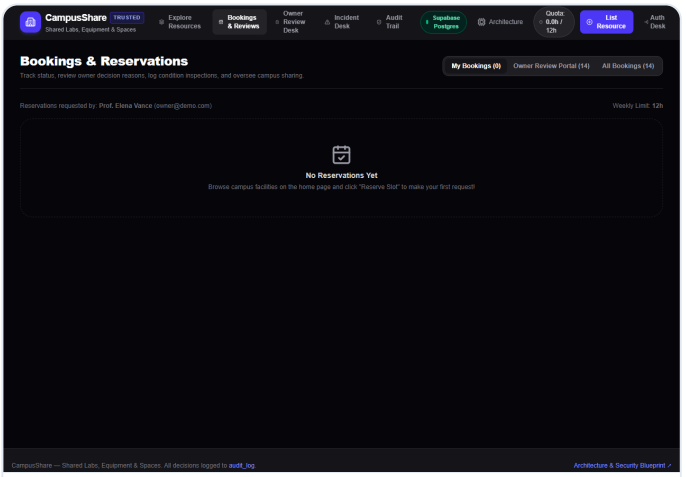


Figure 5: Chronological booking timeline (`/bookings/[id]`) showing full lifecycle verification.

13 Future Scope

- **Multi-Campus Federation:** Enabling cross-institution borrowing between neighboring colleges with mutual EHS safety certification recognition.
- **QR-Code Smart Lock Integration:** Generating time-bounded digital access keys (dynamic QR codes) for automated laboratory door mag-locks upon verified check-in.
- **Supabase Storage Photo Attachments:** Permitting students to photograph and upload timestamped before/after lens and bench condition photos during check-in/return.
- **Full Push & Webhook Notifications:** WebPush and mobile notification webhooks notifying students immediately when a waitlist slot opens.

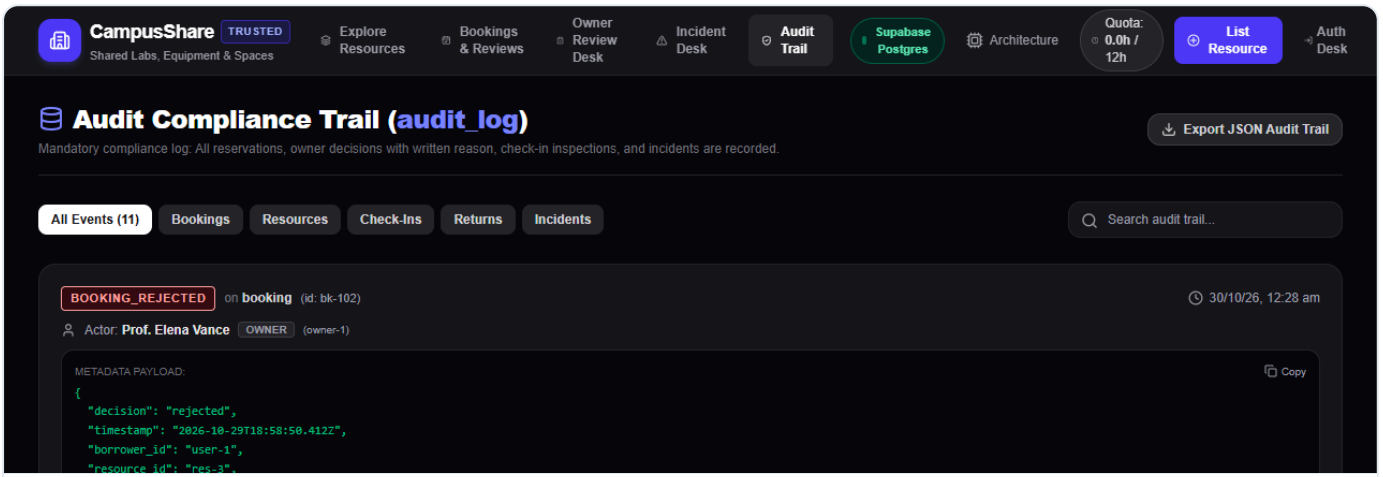


Figure 6: University Compliance Audit Trail (`/audit`) displaying immutable event history, badge filters, and CSV export.

14 Step-by-Step Testing & Evaluation Guide

Judges can test the live deployment on any mobile or desktop browser at: https://hackathon-steel-eta.vercel.app

> **Demo Accounts (Password for all: `DemoPassword123!`):**
> • **Certified Student:** `user1@demo.com` • **Uncertified Student:** `user2@demo.com`
> • **Facility Custodian:** `owner@demo.com` • **Campus Dean:** `admin@demo.com`

1. **Test Certificate Gating:** Sign in as `user2@demo.com`. On the home page, click "Reserve Slot" on *Photonics Lab*. Observe the red alert: `Eligibility Blocked: Missing Prerequisites | You're missing: Laser Safety Tier-2 Certification`. The submit button is hard-disabled.
2. **Test Quota & Booking Request:** Sign in as `user1@demo.com`. Click "Reserve Slot" on *Sony FX6 Camera Kit*. Observe operating hours and 6.0h weekly quota progress. Submit a valid slot. Status becomes `pending`.
3. **Test Owner Review & Justification:** Sign in as `owner@demo.com`. Open `/owner/requests`. Type a reason under 10 characters (e.g. `OK`) → notice live error: `* Needs 8 more characters before submitting` and disabled button. Enter a reason ≥ 10 chars and click Approve.
4. **Test Check-In & Late Return:** Sign in as `user1@demo.com`. Open `/bookings`. Under an active approved session, click "Check In" and submit condition checklist with notes. On return, submit condition rating. If returned late, delay is automatically calculated and logged.
5. **Test Supportive Incident Resolution:** Navigate to `/incidents`. Report an incident (damage, delay, safety). Sign in as `owner@demo.com` and click "Resolve Incident" with a written resolution note.

15 Declaration & Participant Details

Official Hackathon Submission Declaration:
I hereby declare that this project, **CampusShare**, was conceived, architected, developed, and verified during the official hackathon competition window. All code, database schemas, and documentation are original works created by me with the assistance of developer tooling as disclosed in Section 10.

CATEGORY	DETAILS & RESPONSIBILITIES
Participant Name	**MANSURI SUMAN**
College / Institution	**MBIT** (Madhuben & Bhanubhai Patel Institute of Technology)
Team Structure	**Solo Participant**
Roles Executed	• **Product & Research:** Conception of the collegiate trust model, 6.0h weekly quota policy, and non-punitive incident framework. • **UI/UX Design:** Responsive 390px mobile-friendly Tailwind interface, modal interaction flows, and timeline visualizations. • **Backend & Database:** Next.js Server Actions, PostgreSQL schema design, btree_gist exclusion constraints, and Supabase Auth integration. • **QA & Verification:** End-to-end headless browser automation (Puppeteer), role gate security testing, and audit verification.
Personal Contributions	Designed and built the full Next.js 16 application from scratch, authored all server actions, created the PostgreSQL schema with exclusionary constraints, integrated live Supabase database synchronization, resolved production RSC server action revalidation crashes, and conducted comprehensive live testing.
Live Deployment	https://hackathon-steel-eta.vercel.app
Source Code	https://github.com/Sumaan0104/Hackathon

PARTICIPANT SIGNATURE

Mansuri Suman

MBIT • Solo Participant

SUBMISSION DATE

October 30, 2026

Status: Complete • Verified Production Gate